Project Brief: Abundance Ecosystem Events Bot

Project Idea

The goal of this project is to build a lightweight event-discovery and review system for organizations in the “abundance” ecosystem. This includes groups, writers, policy shops, civic organizations, universities, and adjacent movements focused on themes like housing abundance, state capacity, infrastructure, permitting reform, energy, science, water, transportation, governance, and public-sector effectiveness.

Rather than manually checking dozens of organization websites, newsletters, and event pages, the system would regularly monitor a curated list of trusted sources, collect upcoming events, organize them by topic, remove duplicates, and place them into a simple human review queue.

The final product should not feel like a complex scraper. It should feel like a simple dashboard where a less technical user can add organizations, pause sources, update topics, approve events, and publish or export a clean list of upcoming events.

Core Concept

The system should be controlled by editable tables, not hardcoded settings.

A non-technical user should be able to add, remove, pause, or update monitored organizations and tracked topics without needing to touch the underlying code. The scraper or bot simply reads from these tables and follows the current configuration.

The project is best understood as a small “event intelligence pipeline” rather than a one-off web scraper.

It should:

1. Maintain a curated list of organizations and event sources.
2. Check those sources on a regular schedule.
3. Extract event information from the cleanest available source.
4. Normalize the information into a consistent format.
5. Classify events by editable topics.
6. Remove duplicates.
7. Send questionable or new events to a human review queue.
8. Publish approved events to a digest, calendar, website, or shared spreadsheet.

Intended Users

The primary user is someone involved in or tracking the abundance ecosystem who wants to stay informed about relevant events without manually searching every organization’s website.

The secondary user is a less technical administrator or community manager who needs to maintain the system. This person should be able to manage sources and topics through a spreadsheet, Airtable base, or simple dashboard.

The system should be designed so that the technical complexity stays behind the scenes.

Scope

In Scope

The first version should include:

- A curated organization/source registry.
- A topic registry with keywords, aliases, and active/inactive status.
- A daily or weekly bot run that checks active sources.
- Event extraction from organization event pages, RSS feeds, iCal feeds, Luma, Eventbrite, and simple webpage structures.
- Basic AI-assisted extraction and classification where helpful.
- Event normalization into a consistent format.
- Duplicate detection.
- A review queue for human approval.
- Manual edit, approve, reject, and publish workflows.
- Export to a weekly digest, spreadsheet, calendar, or simple public-facing page.

Out of Scope for the First Version

The first version should not try to scrape the entire internet.

It should not rely heavily on social media scraping.

It should not auto-publish all events without review.

It should not require users to write scraping rules.

It should not start with a complex custom frontend unless there is a strong reason to do so.

It should avoid fragile sources such as LinkedIn unless there is an official feed or stable event page available.

Ecosystem Definition

The initial ecosystem would focus on abundance-adjacent organizations and topics, including but not limited to:

- Housing abundance
- YIMBY and zoning reform

- Infrastructure delivery
- Energy abundance
- Transmission and permitting reform
- State capacity
- Civic reform
- Science and innovation policy
- Public-sector effectiveness
- Transit and transportation
- Water supply and water infrastructure
- Government modernization
- Industrial policy and national capacity

The system should allow this definition to evolve. New organizations and topics should be easy to add over time.

System Design

The project should be built around three main tables or data objects:

1. Organizations Table

This table controls what sources the bot monitors.

Suggested fields:

- Organization name
- Website URL
- Events page URL
- Source type
- Category
- Priority
- Active status
- Notes
- Last checked date
- Last successful pull
- Error status

Example fields:

Field	Example
Org name	Institute for Progress
Website	ifp.org
Events page	ifp.org/events
Source type	Website / RSS / Luma / Eventbrite / Calendar

Field	Example
Category	Science, state capacity, policy
Priority	High
Active?	Yes
Notes	Core abundance-adjacent org

Users should be able to pause an organization by changing “Active” from Yes to No. Deactivation should be preferred over permanent deletion so mistakes are easy to reverse.

2. Topics Table

This table controls how events are categorized.

Suggested fields:

- Topic name
- Keywords
- Aliases
- Exclusion terms
- Priority
- Active status
- Notes

Example:

Topic	Keywords	Exclusions	Active?
Housing abundance	housing, zoning, YIMBY, permitting, CEQA, infill	mortgage rates, luxury rentals	Yes
Energy abundance	transmission, clean energy, nuclear, grid, permitting	consumer utility bills only	Yes
State capacity	government reform, implementation, procurement, public sector	general partisan commentary	Yes
Water abundance	water supply, desalination, aqueduct, reservoir, reuse, groundwater	bottled water, consumer filters	Yes

A user should be able to add a new topic, rename a topic, merge topics, or deactivate topics without needing engineering help.

3. Events Table

This table is the human review queue and event archive.

Suggested fields:

- Event title
- Organizer
- Co-organizers
- Start date and time
- End date and time
- Time zone
- Location
- Virtual or in-person
- Registration URL
- Source URL
- Short description
- Full description
- Speakers
- Topics
- Confidence score
- Status
- First seen date
- Last seen date
- Reviewer notes

Suggested event statuses:

- Needs Review
- Approved
- Rejected
- Duplicate
- Published
- Archived

The review queue should be the main place a non-technical user works. They should be able to approve, reject, edit, or merge events.

Event Collection Method

The bot should use the cleanest available source first.

Preferred source order:

1. Official calendar feeds, such as iCal or RSS.
2. Event platforms, such as Luma, Eventbrite, or Meetup.
3. Structured data on the page, such as Schema.org event metadata.
4. Standard HTML event pages.
5. Browser automation for JavaScript-heavy pages.
6. Newsletter or social mentions for discovery only.

The system should favor official event pages over secondary mentions. If an event is mentioned in a newsletter but does not have a clear official registration page, it should be flagged for human review rather than automatically approved.

AI Use

AI should be used as an assistant, not as the source of truth.

Good uses of AI include:

- Extracting event details from messy webpage text.
- Summarizing long event descriptions.
- Classifying events by topic.
- Identifying likely speakers, organizations, and locations.
- Flagging events that seem abundance-adjacent.
- Generating short digest blurbs.

AI should not invent missing details. Dates, titles, locations, and registration links should always be grounded in the original source material.

Duplicate Detection

The system should aggressively detect duplicates because the same event may appear on multiple partner websites.

Duplicate logic should consider:

- Same registration URL.
- Same title and same date.
- Similar title, same speaker, and same city.
- Similar event description and same organizer.
- Same event platform ID.

Possible duplicate matches should be flagged for review rather than automatically deleted.

Review and Publishing Workflow

The recommended workflow is:

1. Bot runs daily or weekly.
2. Bot checks active organizations.
3. Bot extracts possible events.
4. Bot normalizes event fields.
5. Bot classifies each event by topic.
6. Bot checks for duplicates.
7. New or uncertain events go into “Needs Review.”

8. Human reviewer approves, edits, rejects, or merges events.
9. Approved events are published to the selected output.

Possible outputs include:

- Weekly abundance events email.
- Shared Google Sheet.
- Airtable calendar view.
- Public webpage.
- Google Calendar feed.
- Slack or Discord digest.
- CSV export.

Recommended MVP

The recommended first version should be simple.

Use Airtable or Google Sheets as the user-facing control panel. Use a Python script as the backend. Run the script on a schedule. Send approved events to a digest or calendar.

MVP Components

- Airtable or Google Sheets base with three tables:
- Organizations
- Topics

- Events

- Python bot that:

- Reads active organizations.
- Checks event sources.
- Extracts event data.
- Tags events by topic.
- Detects duplicates.

- Writes new events to the review queue.

- Human review process:

- Approve
- Reject
- Edit

- Mark duplicate

- Output:

- Weekly digest or shared calendar of approved events.

Suggested Tech Stack

Lightweight MVP

- Google Sheets or Airtable for administration.
- Python for scraping and processing.
- BeautifulSoup or similar library for HTML parsing.
- Feedparser for RSS.
- iCal parser for calendar feeds.
- Date parsing library for normalizing dates.
- Optional AI API for extraction, classification, and summaries.
- GitHub Actions, cron, or a small hosted job to run on a schedule.

More Developed Version

- FastAPI backend.
- Postgres database.
- Airtable, Retool, or a custom admin interface.
- Scheduled task runner.
- Event review dashboard.
- Public calendar or website frontend.
- Email digest system.

User Experience Requirements

The system should be simple enough that a less technical user can operate it.

They should be able to:

- Add an organization by filling out a form.
- Paste an event page URL.
- Mark a source active or inactive.
- Add or remove tracked topics.
- Edit topic keywords.
- Approve or reject events.
- Correct event details before publishing.
- View which sources are working and which need attention.

The system should avoid exposing technical details unless necessary. Users should not need to understand scraping logic, JSON, source adapters, HTML selectors, or browser automation.

Error Handling and Maintenance

Each source should have a visible health status.

Example statuses:

- Good
- No events found
- Page changed
- Needs attention
- Error checking source
- Last successful pull over 30 days ago

If a source fails, the system should not crash the whole pipeline. It should flag the source and continue checking the others.

The system should also maintain a log of:

- When each source was checked.
- How many events were found.
- Whether parsing succeeded.
- What errors occurred.
- Whether duplicates were detected.

Ethical and Legal Considerations

The system should be designed to be respectful and low-risk.

It should:

- Prefer official feeds and event pages.
- Respect robots.txt and website terms where applicable.
- Avoid bypassing logins, paywalls, or access controls.
- Crawl slowly and cache results.
- Store only event metadata, not full copied articles or private content.
- Link back to original registration pages.
- Avoid heavy scraping of social media platforms.

This should be a helpful monitoring tool, not a high-volume web crawler.

Future Features

After the MVP is working, the project could expand into a more powerful ecosystem intelligence tool.

Possible future features:

- Relationship graph of organizations, events, speakers, topics, and cities.
- Trend dashboard showing which topics are rising.
- Speaker tracking across the ecosystem.
- City-level abundance event map.
- Alerts for high-priority events.

- Automatic weekly or monthly reports.
- “New organization discovery” suggestions.
- Topic heat map.
- Partner organization overlap analysis.
- Public-facing abundance events calendar.
- Personalized event recommendations.

Success Criteria

The project is successful if it reduces manual event tracking while maintaining human control over what gets published.

A successful MVP should:

- Monitor at least 30 to 50 organizations.
- Pull events from multiple source types.
- Let non-technical users manage sources and topics.
- Produce a clean review queue.
- Correctly identify and remove most duplicate events.
- Publish a useful weekly digest or calendar.
- Require minimal engineering work after the initial setup.

Summary

This project should be built as a configurable event-monitoring system for the abundance ecosystem.

The most important design principle is that organizations and topics should be editable by non-technical users. The bot should read from those tables, collect events from trusted sources, classify and deduplicate them, then send them to a human review queue before publication.

The first version should be simple: Airtable or Google Sheets for configuration, a scheduled Python bot for collection, and a review queue for approval. Once that works, the system can grow into a broader map of the abundance movement, showing not only what events are happening, but which ideas, organizations, speakers, and cities are becoming more connected.